



Artbook

Mobile creative marketplace prototype

BACKEND HANDOFF PACK

Artbook Backend Handoff

A practical guide for turning the current phone prototype into a backend-backed app. The APK demonstrates the product behavior; this document explains what must become server-owned, what is fake, and how a backend engineer can start implementing it.

Prototype app

Single-file mobile WebView prototype at `src/artbook-mobile.html`

Installable demo

`artbook-phone-install.apk` for Android testing

Backend scaffold

Plain Node API scaffold in `server/src`

Local API

`http://127.0.0.1:8787` when running

`server/src/server.mjs`

1. Product Scope

What the app is trying to become.

2. Fake vs Real

What must move to backend ownership.

3. Feature Matrix

Screens and required services.

4. API Contract

Endpoints and responsibilities.

5. Data Model

Core entities and relationships.

6. Build Plan

Backend milestones and launch gates.

1. Product Summary

Artbook is a creator-first social, commerce, audio, live, booking and collaboration app. It blends artist profiles, streamer subscription vaults, business services, marketplace listings, events, GPS/manual local discovery, Tour Mode, personal messages, wallet flows and collaborative rooms.

Creator identity

Artists, streamers, businesses, creators and institutions have role-aware profile pages, statuses, privacy settings and studio analytics.

Commerce

Products, services, tickets, digital assets, classes and subscriptions have search, filters, reviews, upload metadata, delivery checks, business-model walk-ins, POS register operations and QR sale paths.

Rooms

Messages, audio rooms, video/audio calls, listen-together sessions and collaboration rooms share a communication model.

Trust

Identity, content rights, delivery route checks, Provenance Seals, forwarding consent and booking protocols are visible in the prototype.

2. Prototype Boundary

Important: the current APK is fake-local. It is good for product review, but it is not a production backend. It uses phone/browser storage and seeded data so all buttons are tappable.

Prototype Behavior	Backend Reality Needed
Local fake wallet balance and PIN 0000 .	Server ledger, immutable transactions, KYC gates, fraud controls, payout provider.
Identity checks instantly approve demo scopes.	KYC provider, document/selfie review, manual escalation, audit logs.
Messages and attachments are local placeholders.	Realtime messaging, object storage, media scanning, abuse reporting.
Calls open simulated audio/video modals.	Call provider such as LiveKit, Daily, Agora or Twilio with permission gates.
Delivery routes are static demo rules.	Courier quote/dispatch/tracking, proof of delivery, disputes and seller payout release.
GPS and locality matching are demo scene matches.	Server geocoding, privacy precision, distance ranking and anti-spoofing controls.
Provenance Seals and reports are local demo records.	Server-owned vouching tied to real orders, bookings, collabs, events, delivery proof and admin review.
Subscriptions and digital buys subtract fake cash.	Google Play Billing entitlements where policy requires, plus server entitlement checks.
Reviews, likes, posts and collabs live locally.	Database models, moderation, notifications, privacy and anti-spam systems.

3. Recommended App Shape

The prototype is strongest if the backend follows five connected workspaces:

Home: feed and urgent work

Profile: public identity and trust

Studio: analytics and tasks

Market: commerce and bookings

Rooms: messages, live, collabs, calls

4. Feature Matrix

Area	Prototype State	Backend Services Needed
Accounts	Switchable demo accounts.	Auth, sessions, MFA, profile ownership, role verification.
Profiles	Status, privacy, role-specific layouts.	Profile API, privacy enforcement, media upload, social links.
Discovery and Tour Mode	Country/city/locality filters, themed pickers, GPS scene matching and scene lenses.	Search index, geocoding, privacy precision, distance ranking, local content feeds.
Social feed	Posts, comments, likes, forwarding policy.	Feed service, moderation, notifications, reporting.
Music and audio	Albums, player, listen-together demo.	Media catalog, streaming URLs, rights metadata, live queue/presence.
Marketplace/POS	Products, services, digital, tickets, classes, vaults, QR sale prompts, anonymous walk-ins, guest intake, split tender, gift cards, staff and drawer controls.	Listings, inventory, reviews, search, QR tokens, counter release codes, prompt verification, business operating model, service-location validation, staff PINs, tender settlement, gift-card liability, cash drawer sessions, orders, disputes.
Provenance Seals	Local vouching, seller score, reports and seller protocol modals.	Relationship-backed seals, risk scoring, reporting queue, payout/listing holds.
Bookings	Calendar, reschedule/cancel protocol.	Booking state machine, reminders, deposits, refund rules.
Messaging	Text, voice/image/video/GIF placeholders.	Realtime chat, media storage, read receipts, blocking/reporting.
Calls	Simulated audio/video modal.	Provider rooms, consultation gate, call logs, safety tools.
Collabs	Rooms, splits, rights ledger, deliverables.	Permission model, shared workspace, contract/rights records.
Wallet	Fake balance, PIN gate, local transactions.	Ledger, KYC, payment provider, Play Billing entitlements, reconciliation.
Delivery	Static route matrix and blocked route examples.	Courier integration, quotes, dispatch, tracking, proof, disputes.

5. Core Data Model

User

Auth identity, email/phone, password hash or OAuth link, session metadata.

Profile

Role, handle, location, status, privacy, verification scopes, media.

Post

Author, text/media, tags, co-creators, forwarding policy, moderation state.

Listing

Owner, kind, price, delivery/resale policies, stock/capacity, review state.

QRSale

Listing/service/ticket token, seller, active/expired state, scan count, prompt link and audit trail.

SalePrompt

Customer ask-ahead queue request, counter-only release code, quantity, window, verification state and handoff proof.

Order

Buyer, items, payment state, delivery state, settlement and disputes.

POSSession

Register/device, staff, drawer open/close, starting cash, paid in/out, counted cash, variance and audit trail.

Tender

Payment method, amount, gift-card/wallet/payment reference, processor status and refund link.

ServiceIntake

Guest reference, consent/allergy/access notes, service place, customer-provided address, privacy level and check-in/out audit.

Booking

Booker/provider, slot, deposit, protocol state, reschedule/cancel events.

Conversation

Members, message permissions, blocks, read state and moderation flags.

Collab

Participants, accepted state, rights ledger, split, deliverables, room notes.

Ledger

Immutable wallet entries, external references, entitlements and audit trail.

TrustSeal

From profile, target profile, type, relationship, evidence, weight, status and audit trail.

TrustReport

Reporter, target, reason, evidence, moderation status, outcome and safety notes.

LocationPreference

GPS/manual mode, country, city, locality, privacy precision, last fix and audit trail.

6. API Contract Summary

The plain Node scaffold in `server/src/server.mjs` demonstrates endpoint shape and server-owned policy decisions. It uses local JSON storage only for development.

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Create user, profile and wallet.
POST	<code>/api/auth/login</code>	Return bearer token.
GET/PATCH	<code>/api/me</code> , <code>/api/profiles/me</code>	Read/update profile, status and privacy.
GET/POST	<code>/api/feed</code> , <code>/api/posts</code>	Fetch and create moderated posts.
GET	<code>/api/discover?</code> <code>country=&city=&locality=&role=&lat=&lng=</code> , <code>/api/locations/resolve</code>	Manual/GPS discovery and server-side scene resolution.
GET/POST	<code>/api/listings</code>	Marketplace listings and review-pending creation.
POST/GET/PATCH	<code>/api/qr-sales</code> , <code>/api/sale-prompts</code>	QR sale token creation, scan resolution, in-shop code release and pickup prompt verification.
POST/PATCH	<code>/api/pos/sessions</code> , <code>/api/pos/tenders/split</code> , <code>/api/pos/gift-cards/redeem</code>	Register sessions, drawer closeout, split tender settlement, gift-card redemption and loyalty accrual.
POST/PATCH	<code>/api/bookings</code> , <code>/api/bookings/:id/reschedule</code> , <code>/api/bookings/:id/cancel</code>	Booking protocol state machine.
POST	<code>/api/messages</code> , <code>/api/media/upload-url</code> , <code>/api/calls</code>	Messaging, media and call room handoff.
POST	<code>/api/payments/intent</code> , <code>/api/orders/checkout</code>	Payment intent and order creation.
POST	<code>/api/delivery/quote</code>	Delivery quote provider adapter.
GET/POST	<code>/api/trust/:profileId</code> , <code>/api/trust/seals</code> , <code>/api/trust/reports</code>	Provenance Seals, seller score and suspicious seller reporting.
POST	<code>/api/collabs</code> , <code>/api/collabs/:id</code>	Collab request and room updates.
POST	<code>/api/data-export</code> , <code>/api/deletion-requests</code>	Privacy rights workflows.

7. QR Pickup Queue

1

Ask

Customer asks ahead. This creates a queue request only, with no release code.

2

Hold

	Seller accepts or declines. Accepted holds reserve stock/capacity until a clear expiry window.
3	Arrive Customer reaches the shop/counter. Seller confirms presence before release.
4	Release Seller creates a short-lived counter code, verifies it, then moves the item into checkout.

8. Booking Protocol

1	Book Booker chooses provider, service and slot. Server creates confirmed booking and deposit/payment state.
2	Reschedule Server records previous slot, new slot, note, actor and whether provider approval is needed.
3	Cancel Server applies cancellation policy, refund percentage, wallet/payment adjustment and notifies both sides.
4	Audit All booking changes are immutable audit events.

9. Trust And Safety Rules

- Money actions must require identity and server-side ledger checks.
- Artist uploads need rights declarations: original work, controlled label catalog or written permission.
- Streamer subscription vaults need identity, age/content policy and consent checks for people shown.
- Business selling requires seller verification, payout setup, refund terms and delivery policy.
- QR sale tokens must be server-owned, expiring and revocable. Customer prompt codes should be created only when the customer is at the shop, then used to release the queued item once.
- Provenance Seals should count only when tied to orders, bookings, collabs, venue proof, delivery proof, identity review or trusted local relationships.
- Low seller score or open reports should trigger listing review, checkout hold, payout delay or manual investigation.
- Forwarding content depends on creator permission, collaboration state and content license.
- Location ranking must respect user privacy: exact GPS, locality, city or hidden.
- Direct business calls are blocked unless caller is a business, artist or streamer; otherwise book consultation.

10. Build Roadmap

1	API and database Move profiles, feed, listings, bookings, messages and collabs to PostgreSQL-backed APIs.
2	Media and realtime Add object storage, signed upload URLs, realtime messaging and room presence.
3	Trust providers Wire KYC, Provenance Seals, seller scoring, moderation, reporting, admin queues and audit logs.
4	Commerce Implement orders, reviews, QR sale prompts with counter-only release codes, tickets, subscriptions, Google Play Billing and payout settlement.
5	Location, delivery and bookings Add geocoding/privacy, delivery quotes/tracking and full booking state machine with refunds.
6	Release engineering Convert to release-grade Android project, add signing/versioning, privacy policy, terms and Play Console assets.

11. Local Developer Commands

Command	Use
<code>node server/src/server.mjs</code>	Run local prototype API.
<code>node server/src/server.mjs --check</code>	Validate the API scaffold imports.
<code>node tools/build-native-artbook-apk.mjs</code>	Rebuild the Android wrapper with location permission support.

Best next step for a backend developer:

Run the server scaffold, read `docs/API_CONTRACT.md`, then choose the first vertical slice: auth + profiles + posts. Once that is server-backed, repeat for marketplace, bookings, messages and collabs.

Play Store note:

The prototype APK is useful for showing the product on a phone. It should not be submitted to Play Store as-is. Before launch, the app needs production backend services, release signing, privacy/data-safety work, Google Play Billing where required, moderation tools and compliance documentation.